

Towards Complete and Scalable Emulation of Quantum Algorithms on High-Performance Reconfigurable Computers — 한국어 해설·정리본

항목	내용
원문 제목	Towards Complete and Scalable Emulation of Quantum Algorithms on High-Performance Reconfigurable Computers
저자	Esam El-Araby, Naveed Mahmud, Mingyoung Jessica Jeng, Andrew MacGillivray, Manu Chaudhary, et al.
게재지	IEEE Transactions on Computers, Vol. 72, No. 8, pp. 2350-2364, August 2023
DOI	10.1109/TC.2023.3248276
분량	15쪽 (본문 p.1-13, 저자 약력 p.13-15). Appendix(Fig. A1-A9, Table A1-A2)는 온라인 보충자료라 이 PDF에 포함되지 않음
원문 파일	../papers/3. QCE-FPGA 논문/Towards_Complete_and_Scalable_Emulation_of_Quantum_Algorithms_on_High-Performance_Reconfigurable_Computers.pdf

페이지 표기 규칙: 이 문서의 원문 p.N 은 모두 **PDF 페이지 번호(1-15)** 입니다. 종이 지면 번호(2350-2364)가 아닙니다. PDF p.N = 지면 2349+N 으로 환산됩니다.

한눈에 보기

FPGA 한 장으로 양자 알고리즘을 **32 큐비트까지** 에뮬레이션한 연구입니다. 기존 FPGA 에뮬레이터가 2-7 큐비트에 머물던 것과 비교하면 자릿수가 다른 규모이고, 그 차이를 만든 것은 화려한 회로 설계가 아니라 **상태벡터를 on-chip이 아니라 on-board DDR4에 두고 스트리밍으로 흘린다**는 단 하나의 결정입니다.

논문의 두 번째 주장은 “complete” 입니다. 기존 에뮬레이터는 “상태가 이미 초기화되어 있다”고 가정하고 알고리즘만 돌렸는데, 이 논문은 고전 데이터를 양자 상태로 실는 **C2Q(classical-to-quantum) 인코딩 회로까지 하드웨어로** 포함시켰습니다. 사례 연구는 양자 하르 변환(QHT)을 이용한 이미지 처리입니다.

우리 프로젝트 관점에서 이 논문의 값어치는 **천장이 어디에 있고 무엇이 그 천장을 만드는지**를 숫자로 알려준다는 데 있습니다. 결론부터 말하면 천장은 로직이 아니라 **메모리**입니다. 32 큐비트에서 FPGA의 LUT는 9.48%, BRAM은 9.29%만 쓰였습니다. 칩의 90%가 비어 있는데도 더 못 올라가는 이유는 진폭이 2^n 개이기 때문입니다.

목차

1. 서론 — NISQ의 한계와 에뮬레이션의 필요성
2. 배경지식
3. 관련 연구
4. 제안 회로 (1) — C2Q 데이터 인코딩
5. 제안 회로 (2) — 다차원·다단계 QHT
6. 에뮬레이션 프레임워크와 하드웨어 아키텍처
7. 실험 환경
8. 실험 결과 — 자원 사용량과 성능
9. 기존 FPGA 에뮬레이터와의 정량 비교
10. 결론과 향후 과제

11. 스케일링 한계 정리 — 무엇이 n 을 묶는가 (해설자 정리)
 12. 이 프로젝트에 어떻게 쓰이나
-

1. 서론 — NISQ의 한계와 에뮬레이션의 필요성

원문 p.1-2

저자들의 출발점은 “지금의 실제 양자 컴퓨터로는 현실적인 회로를 연구할 수 없다”는 문제의식입니다. 현재의 NISQ(Noisy Intermediate-Scale Quantum) 장치에는 세 가지 벽이 있습니다.

결어긋남(decoherence). 큐비트 상태는 환경과 상호작용하며 무너집니다. 회로가 깊으면(depth가 크면) 실행 시간이 길어지고, 그 사이에 노이즈가 누적되어 결과가 틀립니다. 큐비트 수가 적고 연결성이 나쁘면 회로를 넓게(wide) 못 만들고 깊게(deep) 만들 수밖에 없어서, 이 문제가 서로를 악화시킵니다. 따라서 **회로 depth를 줄이는 최적화**가 필수입니다.

C2Q 데이터 인코딩. 이것이 저자들이 특히 강조하는 지점입니다. 양자 알고리즘은 보통 바닥 상태 $|0\rangle^{\otimes n}$ 에서 시작하므로, 실제 데이터를 넣으려면 알고리즘 회로 **앞에** 상태 합성(state synthesis) 회로가 따로 붙어야 합니다. 저자들은 이 과정을 **C2Q(classical-to-quantum) 데이터 인코딩**이라 이름 붙였습니다. 문제는 이 인코딩 회로가 대개 알고리즘 본체보다 복잡하고 깊다는 것입니다. I/O가 많은 실제 응용(이미지 처리 등)에서는 결어긋남 시간 안에 인코딩조차 끝내지 못합니다.

시뮬레이션 비용. 그래서 고전 컴퓨터로 에뮬레이션하게 되는데, 기존 시뮬레이터는 크고 비싸고 하드웨어 가속을 안 쓰는 경우가 많습니다.

이 논문이 내세우는 기여는 다음과 같습니다. FPGA 기반의 **노이즈 없는(noise-free)** 에뮬레이션 프레임워크를 만들되, 여기서 “**complete**”란 상태가 미리 초기화되어 있다고 가정하지 않고 **상태 초기화(C2Q)까지 포함해서** 에뮬레이션한다는 뜻입니다. 사례 연구로 고전 이미지를 양자 회로에 인코딩하고 양자 하르 변환(QHT)을 적용하는 양자 이미지 처리 응용을 통째로 돌립니다. 아키텍처는 **32-bit 부동소수점(floating-point)** 정밀도로, 전 구간 **pipeline** 화하여 throughput을 최대화했습니다. 검증·벤치마크를 위해 CPU 소프트웨어 에뮬레이터와 IBM-Q Qiskit 클라우드 시뮬레이터에도 같은 회로를 구현해 비교했습니다.

저자들은 “양자 상태 초기화와 양자 알고리즘을 통합하여 완전한 양자 이미지 처리 응용의 에뮬레이션을 시연한 최초의 FPGA 기반 에뮬레이션 프레임워크”라고 주장합니다.

2. 배경지식

원문 p.2-3

이 절은 우리 스터디 문서 1-7장과 거의 겹치므로 낱선 것만 짚습니다.

2.1 큐비트 중첩(superposition)과 “비단위 블로흐 구”

큐비트의 순수 상태는 정규화된 벡터입니다 (원문 Eq. 1).

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \equiv \begin{bmatrix} \alpha \\ \beta \end{bmatrix}$$

여기까지는 익숙합니다. 그런데 저자들은 뒤에서 쓰려고 **정규화되지 않은** 임의의 2차원 복소 벡터까지 일반화합니다 (원문 Eq. 2).

$$|\psi\rangle = r e^{i\frac{t}{2}} \left[e^{-i\frac{\phi}{2}} \cos \frac{\theta}{2} |0\rangle + e^{i\frac{\phi}{2}} \sin \frac{\theta}{2} |1\rangle \right]$$

$\theta \in [0, \pi]$ 는 고도각(elevation), $\phi \in [0, 2\pi]$ 는 방위각(azimuth), t 는 **전역 위상(global phase)** 으로 물리적으로 관측 불가능합니다. 핵심은 r 입니다. 보통 정규화된 큐비트라면 $r = 1$ 이지만, 저자들은 $r \neq 1$ 을 허용하고 이를 **전역 스케일(global scale)** 이라 부르며 “반지름이 1이 아닌 블로흐 구”로 해석합니다.

왜 이게 중요한가: $r \neq 1$ 을 허용하면 상태 합성이 훨씬 쉬워집니다. 대신 그 연산자는 **유니터리(unitary)**가 아니게 됩니다. 유니터리가 아니면 실제 양자 하드웨어에서는 못 돌리지만, **에뮬레이터에서는 상관없습니다**. 우리는 그냥 배열에 숫자를 쓰는 것뿐이니깐요. 이 통찰이 뒤의 Method 1을 낳습니다. 즉 (r, t, θ, ϕ) 4-튜플이 이 논문 전체를 관통하는 파라미터화입니다.

2.2 결어긋남

환경과의 상호작용으로 큐비트 상태가 점차 혼합(mixed) 상태가 되며 정보를 잃습니다. 시간이 갈수록 양자 간섭이 억제되고 추가 연산 능력을 잃습니다. 보통 완화 시간 T_1 (바닥 상태로 되돌아가는 시간)과 위상이완 시간 T_2 (환경 노이즈에 굴복하는 시간)로 다룹니다. 회로는 이 시간 안에 끝나야 합니다.

2.3 사용하는 게이트(gate)

아다마르(Hadamard, H), SWAP, CNOT, 회전(Rotation, R_y, R_z), Rotate-Left(RoL), Rotate-Right(RoR)를 씁니다. H 와 SWAP 게이트의 시간 지연을 각각 τ_H, τ_{SWAP} 으로 표기합니다 (뒤의 depth 수식에 등장).

2.4 양자 하르 변환(QHT)

고전 웨이블릿 변환(wavelet transform)은 사인파가 아닌 **모(mother) 웨이블릿**으로 신호를 시공간-스펙트럼 성분으로 분해합니다. 가장 간단한 것이 하르 웨이블릿이고, 단위 계단 함수 $u(t)$ 로 만듭니다 (원문 Eq. 3, a = 시간 신축, b = 변위).

$$\Psi\left(\frac{t-b}{a}\right) = u\left(\frac{t-b}{a}\right) - 2u\left(\frac{t-b}{a} - \frac{1}{2}\right) + u\left(\frac{t-b}{a} - 1\right)$$

이산화하면 (원문 Eq. 4) — 요컨대 **앞 절반 +1, 뒤 절반 -1** 인 사각 펄스입니다.

$$\Psi_D\left(\frac{q-j}{K}\right) = \begin{cases} +1, & 0 \leq (q-j) < \frac{K}{2} \\ -1, & \frac{K}{2} \leq (q-j) < K \\ 0, & \text{otherwise} \end{cases}$$

이산 하르 변환은 (원문 Eq. 5):

$$F_D(j, K) = \sum_{q=0}^{N-1} f_D(q \cdot \Delta t) \Psi_D\left(\frac{q-j}{K}\right)$$

양자 버전에서는 신호 샘플을 중첩된 상태의 **기저 상태 계수(= 진폭)** 로 인코딩합니다 (원문 Eq. 6). 이것이 바로 진폭 인코딩(amplitude encoding)입니다.

$$|\psi\rangle = \sum_{q=0}^{N-1} f(q \cdot \Delta t) |q\rangle, \quad \sum_{q=0}^{N-1} |f(q \cdot \Delta t)|^2 = 1$$

(오른쪽 식이 정규화 조건입니다.)

QHT의 출력은 (원문 Eq. 7):

$$|\psi\rangle_{\text{QHT}} = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} \sum_{q=0}^{N-1} f(q \cdot \Delta t) \Psi_D\left(\frac{q-j}{K}\right) |j\rangle$$

여기서 K 는 웨이블릿 창 크기, n 은 큐비트 수, $N = 2^n$ 은 데이터 샘플 수 = 양자 기저 상태의 총 개수입니다.

직관: 하르 변환의 알맹이는 “이웃한 두 값의 합과 차”입니다. 그런데 합/차를 만드는 게이트가 정확히 **아다마르 게이트**입니다. 그래서 QHT는 순열(permutation) → H 게이트들 → 순열 로 분해됩니다. 무거운 부분은 H가 아니라 **순열**입니다.

2.5 양자 순열(permutation)

QHT의 기본 연산은 순열, 특히 완전 셔플 순열(perfect-shuffle-permutation, PSP)입니다. 양자 순열은 큐비트 순서에 미치는 효과로 기술됩니다. RoL/RoR은 n 큐비트 레지스터에 대한 **순환 좌/우 시프트**이며 (원문 Eq. 8, 9), SWAP 게이트 네트워크로 구현됩니다.

$$\text{RoL}(n-1, 0) : |q_{n-1}q_{n-2} \cdots q_1q_0\rangle \mapsto |q_{n-2} \cdots q_1q_0q_{n-1}\rangle$$

$$\text{RoR}(n-1, 0) : |q_{n-1}q_{n-2} \cdots q_1q_0\rangle \mapsto |q_0q_{n-1}q_{n-2} \cdots q_1\rangle$$

에뮬레이터 관점에서 결정적인 사실: 큐비트 순서를 바꾸는 것은 상태벡터에서 **인덱스 비트를 재배열**하는 것과 같습니다. 즉 순열은 산술 연산이 전혀 아니고 **메모리 주소 계산**입니다. 실제로 뒤에서 저자들은 이걸 곱셈기가

아니라 주소 생성 스케줄러로 구현합니다.

3. 관련 연구

원문 p.3-4

3.1 C2Q 데이터 인코딩의 세 갈래

고전 데이터를 양자 표현으로 옮기는 방법은 크게 셋입니다.

방식	원리	큐비트 비용	문제점
기저 인코딩(basis encoding)	데이터의 이진 표현을 기저 상태로	매우 큼	[25] NE
각도 인코딩(angle encoding)	큐비트 1개당 데이터 1개, 블로흐 구 회전으로	데이터 개수만큼	[26] 이다
진폭 인코딩(amplitude encoding)	데이터를 기저 상태의 진폭(amplitude) 으로	$\lceil \log_2 N \rceil$ — 가장 적음	회로가 복

저자들은 **진폭 인코딩**을 택합니다. $N = 2^n$ 개 데이터를 n 큐비트에 담을 수 있어 큐비트 효율이 압도적이기 때문입니다. 대신 depth가 $O(N)$ 으로 지수적입니다 — 이것이 뒤의 Table 1이 그 큰 지면을 잡아먹는 이유입니다.

기존 연구 중 [27]은 depth $O(n)$ 회로를 제안했지만 큐비트가 $O(N)$ 필요해 현실성이 없습니다. [28]은 depth와 게이트 수를 각각 50% 줄였습니다. [20] Shende 등의 재귀적 방법은 **가장 효율적이라고 알려져 있고 IBM-Q Qiskit의 initialize() API가 실제로 이걸 씁니다**. 이 논문은 [20]과 [28]을 확장해 더 최적화된 방법을 제안합니다. 즉 **비교 대상이 곧 Qiskit의 표준 구현**이라는 구도입니다.

3.2 양자 웨이블릿/하르 변환

Fijany와 Williams [16]가 QWT용 순열 행렬 구현법을 제안했고, [17]은 하르/도비시 웨이블릿의 부분 회로 유도과 다단계·다차원 패킷 QWT 회로를 제시했습니다. 저자들의 지적은 **기존 QWT/QHT 연구가 데이터 초기화 방법도, 회로 최적화도, 실제 하드웨어 구현도 다루지 않았다**는 점입니다. 저자들 자신의 선행 연구 [31]에서 depth 감소 최적화를 제안했고, 이 논문은 거기에 데이터 초기화 통합과 고전 하드웨어 구현을 더한 것입니다.

3.3 GPU 기반 에뮬레이션

GPU 시뮬레이터의 최대 제약은 **크고 비싼 메모리 의존성**입니다. 저자들이 드는 구체적 숫자가 중요합니다.

- **QuEST [33]**: 12 GB GPU로 **29 큐비트**까지. 8 TiB 메모리를 가진 슈퍼컴퓨터에서 **38 큐비트**까지.
- **QIBO [34]**: 16 GB 고급 GPU로 **29 큐비트**까지.
- 여러 서버로 분산하는 고성능 시뮬레이터도 있으나 **서버 간 통신 오버헤드가 무시 못 할 수준**이라 총 실행 시간이 늘어납니다.

여기서 이미 답이 보입니다. 12 GB $\rightarrow$ 29 큐비트, 8 TiB $\rightarrow$ 38 큐비트. 메모리가 **683배** 늘었는데 큐비트는 **9개** 늘었습니다. $2^9 = 512$ 이니 정확히 지수 관계입니다. 큐비트 수는 **메모리 용량의 로그**입니다.

3.4 FPGA 기반 에뮬레이션

기존 FPGA 에뮬레이션 [36]-[42]은 세 가지로 요약됩니다: **낮은 확장성**(적은 큐비트), **낮은 정확도**(고정소수점 정밀도), **낮은 throughput**(낮은 동작 주파수). 게다가 전부 **양자 데이터가 이미 초기화되어 있다고 가정**하고 C2Q 통합이 없어서, 저자들 표현으로는 비현실적이고 “불완전(incomplete)”합니다.

4. 제안 회로 (1) — C2Q 데이터 인코딩

원문 p.4-7

여기가 논문에서 수식이 가장 뻑뻑한 부분입니다. 목표는 하나입니다: **바닥 상태 $|0\rangle^{\otimes n}$ 을 임의의 원하는 상태 $|\psi\rangle$ 로 바꾸는 회로를 만든다.**

$$|\psi\rangle = \sum_{i=0}^{N-1} \alpha_i |i\rangle, \quad \sum_{i=0}^{N-1} |\alpha_i|^2 = 1$$

(원문 Eq. 10. 오른쪽은 정규화 조건입니다.)

저자들은 두 가지 방법을 제안합니다. 이 둘의 차이를 먼저 잡고 가면 이 절 전체가 쉬워집니다.

	Method 1	Method 2
전역 스케일	$r \neq 1$	$r = 1$
연산자	비유니터리(non-unitary)	유니터리(unitary)
실제 양자 장치	불가능	가능
용도	에뮬레이션 전용	물리적 구현 + 에뮬레이션
구조	1단계 균등 제어(uniformly-controlled)	n 단계 피라미드형
상대적 비용	훨씬 싼	비쌈

4.1 Method 1 — 에뮬레이션 전용 비유니터리 상태 합성

원문 p.4-6, Fig. 1-2

임의의 단일 큐비트 비유니터리 게이트는 R_z , R_y 게이트의 열로 분해됩니다. 이를 **ZYZ 분해** 또는 **파울리(Pauli) 분해**라 합니다 (원문 Eq. 11).

$$|\psi\rangle = R_z(\phi) \cdot R_y(\theta) \cdot r e^{i\frac{t}{2}} \cdot |0\rangle = R_z(\phi) \cdot R_y(\theta) \cdot R_z(-t) \cdot r \cdot |0\rangle$$

읽는 법: 바닥 상태 $|0\rangle$ 에 ① **블로흐 구 반지름 스케일링 $r \rightarrow$** ② **z 축 $-t$ 회전 $\rightarrow$** ③ **y 축 θ 회전 $\rightarrow$** ④ **z 축 ϕ 회전** 을 차례로 걸면 임의의 (정규화 안 된) 벡터가 나온다는 뜻입니다.

목표 계수 α, β 가 주어졌을 때 네 파라미터는 이렇게 구합니다 (원문 Eq. 12).

$$r = \sqrt{|\alpha|^2 + |\beta|^2}, \quad t = \angle\beta + \angle\alpha$$

$$\theta = 2 \tan^{-1} \left(\frac{|\beta|}{|\alpha|} \right), \quad \phi = \angle\beta - \angle\alpha$$

여기서

$$|\alpha| = \sqrt{\text{Re}^2(\alpha) + \text{Im}^2(\alpha)}, \quad \angle\alpha = \cos^{-1} \left(\frac{\text{Re}(\alpha)}{|\alpha|} \right)$$

(β 도 동일한 형태입니다.)

전체 상태벡터로 확장하기. 상태벡터의 j 번째 계수 쌍을 합성하려면 바닥 상태에 변환 Δ_j 를 걸면 됩니다 ($j = 0, 1, \dots, 2^{n-1} - 1$). 그런데 문제가 있습니다. n 큐비트 레지스터의 큐비트 하나에 Δ_j 를 걸면 **다른 계수들까지 같이 바뀝니다.** 그래서 각 Δ_j 는 **조건부로** 걸어야 합니다. 결과 회로는 대각 블록이 2×2 행렬 Δ_j 인 **블록 대각 행렬**로 표현됩니다 (원문 Eq. 13).

$$U_{block} = \Delta_0 \oplus \Delta_1 \oplus \dots \oplus \Delta_j \dots \oplus \Delta_{(2^{n-1}-1)} = \text{diag}(\Delta_0, \Delta_1, \dots, \Delta_{(2^{n-1}-1)})$$

블록 대각 행렬은 **균등 제어 회로(uniformly-controlled circuit)** 또는 **양자 멀티플렉서**로 구현됩니다. 균등 제어 회로란 나머지 제어 큐비트들의 **모든 조합 각각에 대해** 타깃 큐비트(여기서는 최하위 큐비트)에 서로 다른 게이트를 거는 회로입니다. n 큐비트 중 $(n - 1)$ 개가 제어, 1개가 타깃입니다. 제어 큐비트의 모든 조합을 **같은 확률로** 만들어야 하므로 U_{block} 앞에 $(n - 1)$ 개의 H 게이트를 겁니다. 전체 변환은 (원문 Eq. 14):

$$|\psi\rangle = U^{C2Q-1} \cdot |0\rangle^{\otimes n}, \quad \text{where } U^{C2Q-1} = U_{block}(r, t, \theta, \phi) \cdot (H^{\otimes(n-1)} \otimes I) = U_0^{C2Q-1}(t, \theta, \phi) \cdot U_{rem}^{C2Q-1}$$

놓치기 쉬운 디테일: $(n - 1)$ 개 H 게이트를 걸었기 때문에 각 블록의 전역 스케일 r_j 는 $2^{(\frac{n-1}{2})}$ 배만큼 **보정**되어야 합니다. 즉 Δ_j 는 파라미터 집합 $\{r_j \cdot 2^{(\frac{n-1}{2})}, t_j, \theta_j, \phi_j\}$ 로 계산됩니다.

각 연산 집합이 서로 배타적이므로 스케일 / z -회전 / y -회전 / z -회전의 **균등 제어 그룹으로 분리**할 수 있고, 이를 ‘사각 상자(square box)’ 표기로 단순화한 것이 원문 Fig. 2입니다 (p.6). 그림을 보면 $n - 1$ 개 제어선 위에 H 가 하나씩 있고, 최하위 큐비트에 $r \cdot 2^{(\frac{n-1}{2})} \rightarrow R_z(-t) \rightarrow R_y(\theta) \rightarrow R_z(\phi)$ 가 순서대로 붙어 있으며, 왼쪽 첫 블록만 “Non-Unitary”로, 나머지 셋은 “Unitary”로 색이 구분되어 있습니다.

핵심: $U_{rem}^{C2Q-1}(r)$ 연산자는 **비유니터리**입니다. 그래서 Method 1은 실제 양자 장치에서 못 돌고 **에뮬레이션 전용**입니다. 대신 아래 Table 1에서 보듯 압도적으로 쌉니다.

4.2 Method 2 — 물리 구현 가능한 유니터리 상태 합성

원문 p.6-7, Fig. 3

전역 스케일을 $r = 1$ 로 만들어 모든 연산자를 유니터리로 만드는 방법입니다. 저자들은 [20] Shende의 재귀적 접근을 **개선**합니다. 결과 회로 U^{C2Q-2} 는 U_j 게이트들의 **피라미드 구조**입니다 ($j = 0, 1, \dots, n-1$).

Shende의 방법은 (원문 Eq. 15):

$$|\psi\rangle = U^{Shende} \cdot |0\rangle^{\otimes n}, \quad U^{Shende} = \left(\prod_{j=0}^{n-1} U_j^{Shende}(\theta, \phi) \otimes I^{\otimes j} \right) \cdot e^{i\frac{t_{n-1}}{2}} = U_0^{Shende}(\theta, \phi) \cdot U_{rem}^{Shende}(\theta, \phi)$$

저자들의 개선 아이디어: Shende의 회로는 R_z 회전이 여러 단계에 흩어져 있고 마지막에 잔여 전역 위상 $e^{i\frac{t_{n-1}}{2}}$ 가 남습니다. 저자들은 **모든 R_z 회전을 첫 단계 U_0 로 몰아넣었습니다.** 그러면 U_0 에만 $R_z(-t), R_y(\theta), R_z(\phi)$ 가 있고, 나머지 $(n-1)$ 개 단계 U_j 에는 $R_y(\theta)$ **회전만** 남습니다. 덤으로 잔여 전역 위상 항도 사라집니다 (원문 Eq. 16).

$$|\psi\rangle = U^{C2Q-2} \cdot |0\rangle^{\otimes n}, \quad U^{C2Q-2} = U_0^{C2Q-2}(t, \theta, \phi) \cdot \left(\prod_{j=1}^{n-1} U_j^{C2Q-2}(\theta) \otimes I^{\otimes j} \right) = U_0^{C2Q-2}(t, \theta, \phi) \cdot U_{rem}^{C2Q-2}(\theta)$$

원문 Fig. 3(c)와 3(d)를 나란히 보면 차이가 한눈에 들어옵니다. (c) Shende는 계단 곳곳에 주황색 R_z 블록이 흩어져 있고, (d) Method 2는 R_z 가 **오른쪽 맨 아래 U_0 한 곳에만** 모여 있고 나머지 계단은 초록색 R_y 뿐입니다.

파라미터 계산 (비재귀적). 각 U_j 는 균등 제어 연산이고, U_j 마다 $k_j = 2^{(n-1-j)}$ 개의 $\Delta_{i,j}$ 회전이 걸립니다. 각 $\Delta_{i,j}$ 에 필요한 2-튜플 $(\alpha_{i,j}, \beta_{i,j})$ 는 아래 점화식으로 구하고, 그것을 Eq. 12에 넣어 4-튜플 $(r_{i,j}, t_{i,j}, \theta_{i,j}, \phi_{i,j})$ 를 얻습니다.

입력 상태벡터를 (원문 Eq. 17)

$$|\psi\rangle = \begin{bmatrix} C_0 \\ C_1 \\ \vdots \\ C_{N-1} \end{bmatrix}, \quad N = 2^n$$

라 할 때, 확률 질량의 부분합 $P_{i,j}$ 를 정의합니다 (원문 Eq. 18).

$$P_{i,j} = \begin{cases} |C_{2i}|^2 + |C_{2i+1}|^2, & j = 0, 0 \leq i < 2^{(n-1)} \\ P_{2i,j-1} + P_{2i+1,j-1}, & 1 \leq j < n, 0 \leq i < 2^{(n-1-j)} \\ 0, & 2^{(n-1-j)} \leq i < 2^{(n-1)} \end{cases}$$

그리고 (원문 Eq. 19, 20):

$$\alpha_{i,j} = \begin{cases} \frac{C_{2i}}{\sqrt{P_{i,j}}}, & P_{i,j} \neq 0, j = 0, 0 \leq i < 2^{(n-1)} \\ \sqrt{\frac{P_{2i,j-1}}{P_{i,j}}}, & P_{i,j} \neq 0, 1 \leq j < n, 0 \leq i < 2^{(n-1-j)} \\ 1, & P_{i,j} = 0 \end{cases}$$

$$\beta_{i,j} = \begin{cases} \frac{C_{2i+1}}{\sqrt{P_{i,j}}}, & P_{i,j} \neq 0, j = 0, 0 \leq i < 2^{(n-1)} \\ \sqrt{\frac{P_{2i+1,j-1}}{P_{i,j}}}, & P_{i,j} \neq 0, 1 \leq j < n, 0 \leq i < 2^{(n-1-j)} \\ 0, & P_{i,j} = 0 \end{cases}$$

($0 \leq j < n, 0 \leq i < k_j, k_j = 2^{(n-1-j)}$)

이 점화식의 의미: $P_{i,j}$ 는 이진 트리의 각 노드에 달린 “확률 질량”입니다. $j = 0$ 은 잎(leaf)에서 이웃 두 진폭의 제곱합, j 가 커질수록 위로 올라가며 두 자식의 합을 취합니다. 그리고 $\alpha_{i,j}, \beta_{i,j}$ 는 “부모 질량 중 왼쪽/오른쪽 자식이 차지하는 비율의 제곱근”입니다. 그래서 자동으로 $r_{i,j} = \sqrt{|\alpha_{i,j}|^2 + |\beta_{i,j}|^2} = 1$ 이 되어 **유니터리성이 보존**됩니다.

중요한 실용적 함의: 이 식들은 **재귀가 아니라 bottom-up 점화식**입니다. Qiskit의 initialize() 는 회로를 **재귀적으로** 구성하는데, 이 재귀가 엄청난 메모리 오버헤드를 낳습니다. 뒤의 결과에서 Qiskit은 16 큐비트에서 메모리 한계에 부딪히는 반면 저자들의 비재귀 Method 2는 20 큐비트까지 갑니다. **알고리즘이 같아도 구현 방식(재귀 vs 반복)이 스케일 한계를 바꿉니다.**

4.3 회로 depth 분석 — Table I

원문 p.7

저자들은 [20]과 같은 방식으로 **1-큐비트 회전 게이트 수와 2-큐비트 CNOT 게이트 수**를 셉니다. CNOT에 집중하는 이유는 **2-큐비트 게이트의 오류율이 단일 큐비트 게이트보다 훨씬 높아서 회로 충실도(fidelity)를 지배하기** 때문입니다.

분해의 기본 단위 (원문 Fig. 4, p.7): n 큐비트 균등 제어 R_y / R_z 회전 하나는 총 2^n 개 게이트로 분해됩니다 — 2^{n-1} 개의 1-큐비트 회전 + 2^{n-1} 개의 2-큐비트 CNOT ($n > 1$). Fig. 4는 3-큐비트 균등 제어 $R_y\{\theta_i\}$ 하나가 $R_y(\hat{\theta}_1)$ -CNOT- $R_y(\hat{\theta}_2)$ -CNOT- $R_y(\hat{\theta}_3)$ -CNOT- $R_y(\hat{\theta}_4)$ -CNOT 로 펼쳐지는 그림입니다.

또한 저자들은 입력 데이터를 **복소 데이터와 양의 실수 데이터**로 구분합니다. 양의 실수 데이터에서는 $\angle\alpha = \angle\beta = 0$ 이므로 Eq. 12에 의해 $t = \phi = 0$ 이 되고, **모든 $R_z(-t), R_z(\phi)$ 게이트가 사라집니다.** 그러면 Method 2와 Shende의 회로가 **완전히 동일**해집니다 (둘 다 n 개 균등 제어 $R_y(\theta)$ 의 피라미드).

Table I-(a) — 게이트 수와 회로 depth (원문 Table I 상단)

방법	항목	복소 데이터(Complex Data)
Shende [20] (기준)	CNOT Gates	$4 \cdot 2^{n-1} - 2n - 2, n \geq 1$
	Total Gates	$8 \cdot 2^{n-1} - 2n - 3$
	Circuit Depth	$8 \cdot 2^{n-1} - 3n - 2$
Method 1 (제안, 에뮬레이션 전용)	CNOT Gates	$0 (n = 1); 2^{n-1} (n > 1)$
	Total Gates	$3 (n = 1); 4 \cdot 2^{n-1} (n > 1)$
	Circuit Depth	$3 (n = 1); 4 \cdot 2^{n-1} (n > 1)$
Method 2 (제안, 유니터리)	CNOT Gates	$2 \cdot 2^{n-1} - 2, n \geq 1$
	Total Gates	$6 \cdot 2^{n-1} - 3$
	Circuit Depth	$6 \cdot 2^{n-1} - n - 2 (1 \leq n \leq 2); 6 \cdot 2^{n-1} - n - 4 (n > 2)$

읽는 법: 모든 식의 지배항이 2^{n-1} 입니다. 즉 **C2Q 회로 depth**는 큐비트 수에 **지수적**입니다. 이것이 진폭 인코딩의 근본적 대가이고, 실제 NISQ 장치에서 C2Q가 걸어긋남을 위반하는 이유입니다. 양의 실수 데이터 열에서 Method 2와 Shende의 값이 **완전히 같다**는 점도 확인하세요.

Table I-(b) — 개선율 $\delta(n)$ (원문 Table I 하단) $\delta(n)$ 은 기준(Shende) 대비 감소 비율이고, $\delta_{max} = \lim_{n \rightarrow \infty} \delta(n)$ 입니다.

비교	항목	복소 데이터 $\delta(n)$	δ_{max}
Method 1 vs Shende	CNOT Gates	$\frac{3 \cdot 2^{n-1} - 2n - 2}{4 \cdot 2^{n-1} - 2n - 2} (n > 1)$	$3/4$
	Total Gates	$\frac{4 \cdot 2^{n-1} - 2n - 3}{8 \cdot 2^{n-1} - 2n - 3} (n > 1)$	$1/2$
	Circuit Depth	$\frac{4 \cdot 2^{n-1} - 3n - 2}{8 \cdot 2^{n-1} - 3n - 2} (n > 1)$	$1/2$
Method 2 vs Shende	CNOT Gates	$\frac{2 \cdot 2^{n-1} - 2n}{4 \cdot 2^{n-1} - 2n - 2} (n > 1)$	$1/2$
	Total Gates	$\frac{2 \cdot 2^{n-1} - 2n}{8 \cdot 2^{n-1} - 2n - 3}$	$1/4$
	Circuit Depth	$\frac{2 \cdot 2^{n-1} - 2n}{8 \cdot 2^{n-1} - 3n - 2} (1 \leq n \leq 2); \frac{2 \cdot 2^{n-1} - 2n + 2}{8 \cdot 2^{n-1} - 3n - 2} (n > 2)$	$1/4$

($n = 1$ 인 경우는 모든 칸에서 $\delta = 0$ 입니다.)

결론 세 줄. 복소 데이터에서 Method 2는 Shende 대비 **총 게이트 수와 회로 depth**를 **점근적으로 25% 감소**시키고, **2-큐비트 CNOT 게이트를 50% 감소**시킵니다. 양의 실수 데이터에서는 두 방법이 동일하므로 개선율이 **0**입니다. 에뮬레이션 전용 Method 1은 CNOT을 최대 **75%** 까지 줄입니다. 저자들은 이 이론적 예측을 IBM-Q Qiskit API로 실험 검증했고 “완벽히 일치(perfect match)”했다고 보고합니다.

5. 제안 회로 (2) — 다차원·다단계 QHT

원문 p.7-8

d 차원 QHT는 고전 웨이블릿 변환처럼 l 단계로 분해 가능하며, **패킷(packet)** 방식과 **피라미드(pyramidal)** 방식이 있습니다. 저자들은 데이터 차원이 소실되지 않는 **무손실 패킷 분해**를 씁니다. 패킷 분해에서는 전 과정에 모든 데이터 큐비트가 필요합니다. 최대 분해 단계 수 l_{max}^{pkt} 는 차원 수 d 와 총 큐비트 수 n 에 의존하되, **무손실 분해의 최대 단계 수는 모든 d 개 차원에 걸친 큐비트 수의 최솟값과 같습니다.**

d 차원 QHT 연산 $U^{d-D-QHT}$ 는 세 부분으로 구성됩니다:

1. 상태벡터에 적용되는 **입력 순열(input permutation)**
2. **하르 변환 연산 (H 게이트들)**
3. 출력 상태벡터를 만드는 **출력 순열(output permutation)**

구현 방식은 **병렬(1-stage)** 과 **순차(d -stage)** 두 가지이며, 이 논문은 병렬(1-stage) 최적화를 다룹니다.

5.1 최적화 전 vs 최적화 후

최적화 전에는 H 연산이 병렬로 한 번에 걸리고, RoR/RoL 연산이 앞뒤에 각각 뭉쳐 있습니다. 총 지연 시간은 (원문 Eq. 21):

$$t_{total}^{par, unopt, pkt} = \left((2n - n_{(d-1)} - (2d - 1)) \cdot \tau_{SWAP} + \tau_H \right) \cdot l$$

주의: 여기 $n_{(d-1)}$ 은 **아래첨자**입니다 ($(d - 1)$ 번째 차원의 큐비트 수). PDF 텍스트 레이어를 복사하면 $n \cdot (d - 1)$ 처럼 보이는데 곱셈이 아닙니다. 페이지 이미지를 직접 확인한 결과입니다.

최적화 아이디어: H 게이트를 n_i 큐비트만큼 떨어뜨려 **재배치**합니다 ($0 \leq i < d$). 그러면 **앞쪽 순열(RoL 게이트)이 아예 필요 없어지고**, 뒤쪽 RoR 연산도 depth가 줄어들며 서로 독립이므로 **병렬 적용**이 가능합니다. 총 지연 시간은 (원문 Eq. 22):

$$t_{total}^{par, opt, pkt} = ((n_{max} - 1) \cdot \tau_{SWAP} + \tau_H) \cdot l$$

여기서 n_{max} 는 모든 차원에 걸친 최대 큐비트 수, l 은 분해 단계 수입니다.

효과: 지연이 $(2n - n_{(d-1)} - (2d - 1))$ 에서 $(n_{max} - 1)$ 로 줄었습니다. $n = \sum n_i$ 이므로 차원이 많아질수록(d 가 클수록) 이득이 커집니다. 예를 들어 3차원에서 각 차원이 균등하게 $n/3$ 큐비트씩이라면 $n_{max} = n/3$ 이므로 대략 $2n \rightarrow n/3$, 즉 **6배 가까이** 짧아집니다. 게이트를 없앤 게 아니라 **위치를 바꿔서 순열을 제거하고 나머지를 병렬화**한 것이 요점입니다.

6. 에뮬레이션 프레임워크와 하드웨어 아키텍처

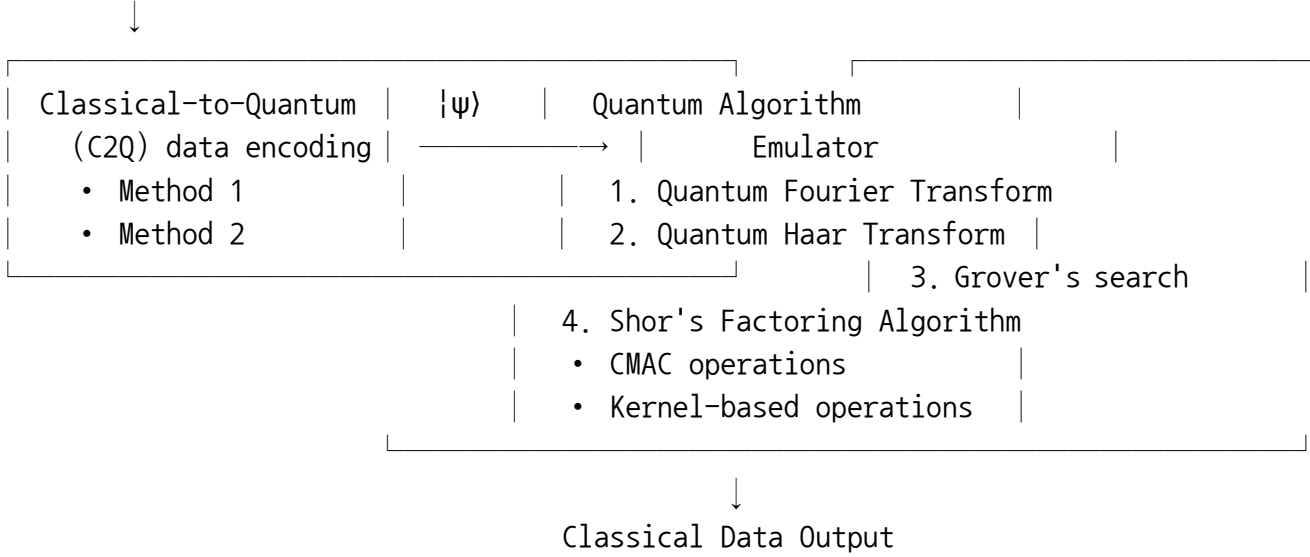
원문 p.8-9

우리 프로젝트에 가장 중요한 절입니다.

6.1 프레임워크 전체 구조 (원문 Fig. 5, p.8)

Fig. 5는 놀랍도록 단순한 2-블록 파이프라인입니다.

Classical Data Input



여기서 반드시 챙길 점 두 가지.

첫째, 그림에 **Grover's search**가 대상 알고리즘 3번으로 명시되어 있습니다. 이 프레임워크는 QHT 전용이 아니라 그로버를 염두에 둔 범용 골격입니다.

둘째, 저자들은 알고리즘 종류에 따라 **에뮬레이션 기법**을 두 갈래로 나눕니다. 이것이 이 논문에서 우리 프로젝트에 가장 직접적인 설계 지침입니다.

변환 행렬의 성질	해당 알고리즘	에뮬레이션 기법
조밀(dense) 행렬	양자 푸리에 변환(QFT), 그로버 검색(Grover's search)	CMAC(complex-multiply-and-accumulate)
희소(sparse) 변환 행렬	QHT	커널 기반(kernel-based) 연산 [31]

즉 그로버는 이 논문이 실제로 구현한 QHT와 반대편 진영입니다. 그로버용 CMAC 아키텍처는 이 논문이 아니라 참고문헌 [43] (Mahmud, El-Araby, Caliga, "Scaling reconfigurable emulation of quantum algorithms at high precision and high throughput," Quantum Engineering, vol. 1, no. 2, 2019, Art. no. e19)에 있습니다. 그로버 에뮬레이터를 만든다면 [43]이 다음에 읽을 논문입니다.

6.2 C2Q 커널 아키텍처 (원문 Fig. 6, p.8)

단일 큐비트의 목표 상태를 $|0\rangle$ 에서 합성할 때, Eq. 11의 파울리 분해로 복소 계수 쌍 (α, β) 를 4-튜플 (r, t, θ, ϕ) 로 표현할 수 있습니다 (원문 Eq. 23):

$$\alpha = r \cdot e^{i\frac{t-\phi}{2}} \cdot \cos\left(\frac{\theta}{2}\right), \quad \beta = r \cdot e^{i\frac{t+\phi}{2}} \cdot \sin\left(\frac{\theta}{2}\right)$$

Fig. 6은 이 수식을 그대로 하드웨어로 옮긴 순수 데이터패스입니다. 왼쪽에서 $r_j, \theta_j, \phi_j, t_j$ 가 들어오고, $\gg 1$ 시프트 블록들이 각 각도를 반으로 나누고($\theta/2, (t \pm \phi)/2$), cos/sin 블록이 삼각함수를 계산하고, 가산기와 곱셈기 트리가 이를 조합해 오른쪽으로 $\alpha_j^{real}, \alpha_j^{imag}, \beta_j^{real}, \beta_j^{imag}$ 네 개의 실수를 뱉습니다. 상태 저장도, 반복 루프도, 제어 로직도 없는 **완전 조합형 파이프라인**입니다.

전체 N 개 상태의 상태벡터를 합성하려면 이 커널이 **pipeline 방식으로 반복 동작**하면서 Fig. 2(Method 1) 또는 Fig. 3(Method 2)의 회로 구조를 만들어 나갑니다. 즉 중간 입력 파라미터 집합 $r_j, t_j, \theta_j, \phi_j$ ($j = 0, 1, \dots, \frac{N}{2} - 1$)로부터 중간 상태벡터의 복소 계수 $\frac{N}{2}$ 쌍을 합성합니다.

설계 철학이 여기 다 드러납니다. 커널은 작고 멍청하고 상태가 없습니다. 대신 데이터가 그 위로 흐릅니다. 회로의 크기가 n 에 따라 커지는 게 아니라, **같은 커널에 흘려보내는 데이터의 양이 2^n 으로 커집니다.** 이것이 32 큐비트를 가능하게 한 구조적 결정입니다.

6.3 QHT 커널 아키텍처 (원문 p.9)

3D-QHT의 세 구성요소는 각각 P_{in}^{3D} (입력 순열), $U^{3-D-QHT}$ (하르 변환), P_{out}^{3D} (출력 순열)입니다.

입력 순열 P_{in}^{3D} 는 하드웨어 스케줄러로 모델링됩니다. 동작은 세 단계입니다: 입력 상태벡터를 메모리에서 읽고 $\rightarrow$ 데이터 포인트의 **새 인덱스를 생성**하고 $\rightarrow$ 생성된 인덱스로 메모리에 되쓰면서 출력 상태벡터를 만듭니다. 원문은 $(4 \times 4 \times 4)$ 픽셀 3D 이미지를 나타내는 상태 $|x\rangle$ 가 P_{in}^{3D} 를 거치는 예를 Fig. A7로 보여줍니다(온라인 보충자료).

즉 순열에는 곱셈기가 하나도 없습니다. 앞서 2.5절에서 예고한 대로, 양자 순열은 산술이 아니라 **주소 재계산**입니다. 이것이 저자들이 QHT를 “최소 행렬” 진영으로 분류하고 CMAC 대신 커널 기반 연산을 쓰는 이유입니다.

순열된 상태에서 **3D 하르 변환은 한 번에 픽셀 8개 묶음($2 \times 2 \times 2$)에 적용**됩니다. 출력 순열 P_{out}^{3D} 의 동작과 아키텍처는 P_{in}^{3D} 와 유사합니다.

7. 실험 환경

원문 p.9-11

세 가지 구현을 나란히 놓고 비교합니다: (a) 재구성 가능 시스템(CPU + FPGA), (b) CPU 단독, (c) IBM-Q Qiskit 클라우드 시뮬레이터.

7.1 FPGA (HPRC) 환경

플랫폼: Xilinx Alveo U250 Data Center Accelerator + 호스트 머신.

구분	사양
호스트 CPU	16-core, 3 GHz AMD CPU
호스트 메모리	251 GB
인터커넥트	PCIe Gen 3 (호스트↔보드 구성 및 데이터 통신)

구분	사양
FPGA	XCU250 (Xilinx SSI 기술, 4개의 SLR (super logic region) 결합)
동적 영역 LUT	1,341K
동적 영역 레지스터	2,749K
동적 영역 BRAM	2,000 × 36 KB
동적 영역 DSP	11,508 slices
on-board 메모리	16 GB 288-pin DDR4 DIMM × 4소켓 (single rank)
메모리 전송률	최대 2,400 MegaTransfers/s
툴체인	Xilinx Vitis Unified Software, OpenCL (커널 + 호스트 프로그램), MATLAB R2020a (전처리/후처리)

FPGA는 **정적 영역(static region)** 과 **동적 영역(dynamic region)** 으로 나뉩니다. 정적 영역에는 PCIe로 장치 bring-up과 설정을 담당하는 **deployment shell**(OpenCL shell)이 들어 있고, 나머지 동적 영역이 개발자가 커스텀 가속기와 커널을 구현하는 공간입니다.

7.2 실행 모델과 타이밍 (원문 Fig. 7, p.9)

Fig. 7(a)는 FPGA 상의 커널 배치를 보여줍니다: 호스트 ↔ (PCIe) ↔ 보드 on-board memory ↔ 두 개의 커널. `kernel_c2q` 로 4-튜플 (r, t, θ, ϕ) 가 들어가고 $|\psi\rangle_{in}$ 이 나와 메모리에 저장되며, 그 $|\psi\rangle_{in}$ 이 `kernel_qht` 로 들어가 $|\psi\rangle_{out}$ 이 나옵니다. Fig. 7(b)는 CPU / 호스트 메모리 / (PCIe) / 가속기 on-board 메모리 / FPGA 사이의 데이터 이동과 각 구간의 측정 시간을 보여줍니다.

시간 항목의 정의:

기호	의미
T_{setup}	호스트의 메모리 할당, 커널 객체·큐 설정 시간
T_{config}	PCIe로 FPGA를 프로그래밍(configure)하는 시간
T_{in}	호스트 메모리 → FPGA on-board 메모리 전송 시간
T_{comp}	FPGA 커널의 연산 시간 (FPGA ↔ on-board 메모리 전송 시간 포함)
T_{out}	on-board 메모리 → 호스트 메모리 전송 시간
T_{CPU}	호스트가 연산을 수행하는 전체 시간 (호스트 메모리 전송 포함)

$$T_{FPGA} = T_{in} + T_{comp} + T_{out}$$

T_{setup} 과 T_{config} 는 CPU 실험과의 형평을 위해 분석에서 제외되었습니다. 즉 보고된 FPGA 시간에 비트스트림 로딩 시간은 포함되지 않습니다.

실행 흐름: 입력 데이터셋에서 4-튜플 (r, t, θ, ϕ) 를 추출하는 작업은 **호스트 머신**이 합니다. 파라미터와 입출력 상태벡터 $|\psi\rangle_{in}, |\psi\rangle_{out}$ 은 **on-board 메모리에 저장**되고 연산 시 커널의 재구성 영역으로 전송됩니다. 호스트가 PCIe로 메모리 전송과 커널 실행 명령을 제어합니다. `kernel_c2q` 가 먼저 실행되어 $|\psi\rangle_{in}$ 을

합성하고, 그것이 kernel_qht 로 넘어가 병렬 l -단계 d -차원 QHT를 수행해 $|\psi\rangle_{out}$ 을 만들어 on-board 메모리로 되돌립니다.

커널 구현 사양: 전 구간 **fully pipelined**, 연산은 **32-bit 부동소수점(floating-point) 산술**.

입력 데이터: $(16 \times 16 \times 4)$ 픽셀부터 $(32,768 \times 32,768 \times 4)$ 픽셀까지의 multi-spectral 이미지. 이 이미지들에 대해 **10 ~ 32 큐비트**의 C2Q 및 3D-QHT 회로를 에뮬레이션했습니다. 같은 크기의 복소 데이터는 시드가 고정된 난수 생성기로 만들었습니다. 호스트-FPGA 대역폭을 완전히 활용하기 위해 **데이터 패킹(data packing)** 기법을 사용해 최적의 전송·연산 시간을 달성했습니다.

큐비트 수의 유래: $(32,768 \times 32,768 \times 4) = 2^{15} \times 2^{15} \times 2^2 = 2^{32}$ 개 원소이고, 이것이 곧 $N = 2^{32}$, 즉 $n = 32$ 큐비트입니다. 이 숫자를 기억해 두세요. 11절에서 다시 나옵니다.

7.3 CPU 환경

칸자스 대학(KU)의 HPC 클러스터. 각 노드는 **2 × 12-core Intel Xeon E5-2680 v3** (base clock 2.50 GHz), PCIe Gen 3.0, **503 GB 메모리**(8 × 64 GB DDR4 DIMM @ 2,133 MHz)입니다. 제안 아키텍처의 소프트웨어 에뮬레이터를 **C++** 로 작성했고, FPGA 실험과 동일한 양의 실수(multi-spectral 이미지) 및 복소 데이터를 사용해 **10 ~ 32 큐비트**를 에뮬레이션했습니다. 4-튜플 파라미터와 $|\psi\rangle_{in}, |\psi\rangle_{out}$ 은 heap에 할당했습니다.

중요: CPU 측정은 호스트 CPU의 **단일 코어(single-core)** 실행에서 취했습니다. 입력 텍스트 파일 읽기 시간은 제외했습니다.

이 점은 벤치마크를 해석할 때 감안해야 합니다. 24코어 노드에서 1코어만 쓴 결과이므로, 뒤에 나올 FPGA의 $\times 21.66$ 같은 speedup은 **완전히 병렬화한 CPU 대비가 아닙니다**.

7.4 IBM-Q Qiskit 환경

CPU 실험과 **같은 시스템**에서 **Qiskit SDK (v0.38)** 기반 노이즈 없는 클라우드 시뮬레이터를 사용했습니다. 결과는 하루 중 다른 시간대에 **최소 10회 이상 실행한 값의 중앙값(median)** 으로, 클라우드 큐 대기의 시간 의존성을 상쇄하기 위함입니다.

비교 대상은 Qiskit의 qiskit.circuit.QuantumCircuit.initialize() 메서드로, [20] Shende 등의 연구에 기반합니다. 모든 C2Q 회로는 동일한 입력 데이터를 받아 **2-큐비트 CNOT + 단일 큐비트 회전 게이트로 트랜스파일(transpile)** 되었습니다. 바닥 상태에서 입력 상태로 초기화한 뒤 3D-QHT를 적용했습니다. 보고된 결과는 job.result() API를 통한 **1 shot** 실행이며, 회로 구성·트랜스파일·어셈블 시간은 제외됩니다.

한계점 (핵심 숫자): initialize() API를 쓴 Qiskit 트랜스파일은 **16 큐비트**를 넘는 C2Q 회로에서 시스템 메모리 한계를 초과했고, 3D-QHT는 **28 큐비트**가 한계였습니다.

7.5 데이터 시각화 (원문 Fig. 9, p.11)

Fig. 9(a)는 실험에 사용한 $(64 \times 64 \times 3)$ 픽셀 RGB 샘플 입력 이미지입니다. 입력 이미지는 **스펙트럼 차원을 2의 거듭제곱으로 만들기 위해 0으로 패딩**되었습니다($3 \rightarrow 4$). 이미지 데이터는 MATLAB에서 1차원 벡터로 변환·정규화되고, C2Q에 필요한 파라미터가 추출되어 FPGA/CPU/IBM-Q 시뮬레이터에 입력됩니다.

역정규화, 패딩 제로 제거, 이미지 재구성 같은 후처리로 MATLAB에서 수행합니다. Fig. 9(b)는 1-level 3D-QHT를 거친 후 재구성된 출력 이미지로, 분해 후 각 차원의 크기가 $\frac{1}{2^l}$ 배로 줄어듭니다 (l = 분해 단계 수). 결과적으로 이미지 내용은 보존되면서 해상도만 절반이 된, 올바른 차원 축소가 시각적으로 확인됩니다.

원문 그림 라벨 주의: PDF와 대조할 때 헷갈릴 만한 지점이 둘 있습니다. 첫째, Fig. 8에서 **(b)와 (c)의 그래프 이미지가 캡션과 서로 뒤바뀌어** 실려 있습니다. 캡션 (b)“C2Q complex data” 자리에 실제로는 제목이 “Quantum Haar Transform (QHT) using Multi-Spectral Images” 인 QHT 그래프가, 캡션 (c)“3D-QHT” 자리에 제목이 “Classical-to-Quantum (C2Q) for Complex Data” 인 그래프가 있습니다. **그래프 안의 제목을 믿으세요.** 둘째, Fig. 9의 캡션은 “1-level parallel (1-stage) 2D-QHT”라고 되어 있으나 본문은 “1-level 3D-QHT”라고 씁니다.

8. 실험 결과 — 자원 사용량과 성능

원문 p.10-12

8.1 FPGA 자원 사용량 — Table II (원문 p.10)

이 표가 이 논문에서 가장 중요한 표입니다. post-place-and-route 기준 사용량입니다. FPGA 면적은 호스트 인터페이스와 메모리 전송을 담당하는 **정적 영역(OpenCL shell)** 과 C2Q·3D-QHT 커널이 들어가는 **재구성 영역**으로 구성됩니다.

Resource	Static OverlayC2Q (Method 1)	Static OverlayC2Q (Method 2)	C2Q Kernel(Method 1)	C2Q K
LUT	110,382 (6.39%)	138,410 (8.02%)	4,688 (0.29%)	11,93
LUTAsMem	15,639 (1.98%)	21,035 (2.78%)	746 (0.1%)	1,174
REG	175,665 (5.1%)	250,988 (7.26%)	5,321 (0.16%)	12,05
BRAM	203 (7.6%)	228 (8.48%)	3 (0.12%)	18 (0.
DSP	4 (0.03%)	7 (0.06%)	24 (0.2%)	40 (0.

이 표에서 반드시 읽어야 할 것들.

커널 자체는 믿기 힘들 만큼 작습니다. 3D-QHT 커널 전체가 LUT **11,478개(0.71%)**, BRAM **2개(0.08%)**, DSP **27개(0.22%)** 입니다. C2Q 커널(Method 1)은 LUT 4,688개(0.29%), **BRAM 3개**, DSP 24개입니다. **BRAM 2~3개로 32 큐비트를 돌립니다.** 2^{32} 개 진폭이 BRAM에 있을 리 없죠 — 전부 **on-board DDR4**에 있고 BRAM은 스트리밍 버퍼 역할만 합니다. 이것이 이 논문의 마술 전체입니다.

대부분의 자원은 커널이 아니라 **shell**이 씁니다. Method 2 기준 총 LUT 161,819개 중 **138,410개(85%)**가 정적 영역인 **OpenCL shell**입니다. 즉 사용자가 짠 양자 에뮬레이션 로직보다 PCIe/메모리 인프라가 10배 이상 큼니다.

전체 사용률이 10% 미만입니다. 최대치가 LUT **9.48%**, BRAM **9.29%** 입니다. 저자들의 결론: “Therefore, more emulation engines, up to $\times 10$, of the hardware kernels can be instantiated on a single FPGA to achieve higher throughput and faster emulation times.” — 즉 커널을 **$\times 10$ 복제**해 throughput을 높일 수 있습니다.

이 $\times 10$ 주장을 정확히 이해해야 합니다. 커널 10개를 복제하면 더 빨라지지만 큐비트가 늘어나지는 않습니다. 큐비트 수를 결정하는 것은 메모리 용량이지 커널 개수가 아닙니다. 10개 엔진이 같은 16 GB DIMM을 공유한다면 오히려 메모리 대역폭 경쟁으로 이득이 줄어들 수도 있습니다. 이 논문은 실제로 $\times 10$ 복제를 구현하거나 측정하지 않았습니다 — **미래 가능성에 대한 주장입니다.**

동작 주파수: 411 MHz ~ 414 MHz (Table III 기준, C2Q 414 MHz / QHT 411 MHz).

8.2 FPGA vs CPU 성능 (원문 Fig. 8, p.10)

Fig. 8은 큐비트 수 10~32에 대한 로그 스케일 실행 시간(ms) 그래프 3장입니다.

교차점(crossover). CPU가 C2Q는 12 큐비트까지, 3D-QHT는 16 큐비트까지 FPGA보다 빠릅니다. 이유는 CPU와 호스트 메모리 서브시스템이 **데이터 캐싱**의 이득을 볼 수 있기 때문입니다. 그러나 데이터 크기와 회로가 커지면 캐싱이 무력화(throttled)되고, FPGA가 **높은 대역폭과 fine-grain 병렬성**의 이득을 보기 시작해 역전합니다.

그래프 모양을 말로 옮기면 이렇습니다. 로그 스케일 세로축에서 모든 곡선이 **직선**입니다. 로그 축에서 직선이라는 건 실행 시간이 2^n 에 정비례한다는 뜻, 즉 $O(2^n)$ 입니다. 작은 n 에서는 CPU 선이 아래(빠름)에 있다가, 교차점을 지나면 FPGA 선이 아래로 내려가 32 큐비트까지 **일정한 간격**을 유지하며 **평행하게** 갑니다. 평행하다는 것이 중요합니다 — **FPGA는 지수 곡선의 상수 계수를 낮출 뿐, 기울기(지수성) 자체를 바꾸지 못합니다.**

Speedup (FPGA 총 실행 시간 / CPU 총 실행 시간 기준):

비교	데이터	Speedup
C2Q (Method 1) FPGA vs CPU	양의 실수(multi-spectral 이미지)	$\times 21.66$
3D-QHT 커널 FPGA vs CPU	양의 실수(multi-spectral 이미지)	$\times 3.49$
C2Q (Method 2) FPGA vs CPU	복소 데이터	$\times 1.34$

저자들 스스로 인정하듯, Method 2의 FPGA 에뮬레이션은 **deep pipelining, loop fusion, superscaling, dense data packing/unpacking** 같은 추가 하드웨어 최적화의 여지가 남아 있으며 이는 향후 연구 과제입니다. ($\times 1.34$ 는 사실상 무승부에 가까운 숫자입니다.)

8.3 FPGA vs Qiskit 성능 (원문 p.12)

스케일 한계 비교 — 이 논문의 핵심 주장입니다.

구현	최대 큐비트
Qiskit Initialize() API 사용 C2Q 회로	16
제안 Method 2 사용 C2Q 회로 (Qiskit 상에서)	20
Qiskit QHT 회로	28
제안 FPGA 에뮬레이션	32

Qiskit은 500 GB 이상의 시스템 메모리를 쫓는데도 위 한계에 묶였습니다.

Speedup:

- 20 큐비트 C2Q 회로: FPGA가 Qiskit 시뮬레이터 대비 4자릿수(4 orders of magnitude) 이상의 speedup.
- 26 큐비트 3D-QHT 회로: FPGA가 Qiskit 대비 최대 $\times 148.33$.

8.4 Method 2 vs Qiskit 회로 depth (원문 Fig. 10, p.12)

양의 실수(multi-spectral 이미지) 데이터에서는 두 기법이 동일한 회로를 합성하여 회로 depth와 총 게이트 수가 완전히 같았습니다. 그러나 복소 데이터에서는 제안 Method 2가 최대 25%의 회로 depth 개선을 보였습니다.

Fig. 10은 큐비트 1~20에 대한 depth 감소율(%) 그래프입니다. 이론값(Theoretical)과 실험값(Experimental) 두 계열이 완벽히 겹치며, $n = 1$ 에서 0%로 시작해 $n = 2$ 에서 급히 치솟아 $n \approx 4$ 부터 25% 근처로 점근하는 포화 곡선입니다. 이는 Table I의 $\delta_{max} = 1/4$ 예측과 정확히 일치합니다. 저자들 표현으로 실험 측정이 이론 기대치와 “perfectly match”합니다.

또 하나의 발견 — 메모리 오버헤드. Qiskit의 Initialize() 는 시뮬레이션 시 기본 동작이 C2Q 회로를 구성하는 게 아니라 상태벡터를 원하는 값으로 직접 초기화하는 것입니다. 회로 depth를 측정하기 위해 저자들은 Method 2와 Initialize() 양쪽 모두 Qiskit 트랜스파일러를 태워 실제로 C2Q 회로를 합성시켰습니다. 그 결과 Qiskit API는 16 큐비트에서, Method 2는 20 큐비트에서 메모리 한계에 도달했습니다. 저자들의 해석은 Initialize() 가 C2Q 회로를 재귀적으로 구성하기 때문에 비재귀적인 Method 2 (Eq. 17-20) 대비 훨씬 큰 메모리 오버헤드를 유발한다는 것입니다.

9. 기존 FPGA 에뮬레이터와의 정량 비교

원문 p.12-13, Table III

이 표에 그로버 항목이 세 개 있습니다. 우리 프로젝트의 좌표를 정확히 찍어주는 표입니다.

Reported Work	Algorithm	Number of qubits	Precision	FPGA device
Fujishima (2003) [36]	Shor's factoring	N/A	N/A	Altera APEX
Khalid et al. (2004) [37]	QFT	3	16-bit fixed pt.	Altera Stratix
Khalid et al. (2004) [37]	Grover's search	3	16-bit fixed pt.	Altera Stratix
Aminian et al. (2008) [38]	Grover's search	3	16-bit fixed pt.	Altera Stratix
Lee et al. (2016) [39]	QFT	5	24-bit fixed pt.	Altera Stratix
Lee et al. (2016) [39]	Grover's search	7	24-bit fixed pt.	Altera Stratix
Silva and Zabaleta (2017) [40]	QFT	4	32-bit floating pt.	AMD Xilinx Zynq
Pilch and Dlugopolski (2018) [41]	Deutsch	2	N/A	Altera Cyclone
Suzuki et al. (2022) [42]	Image classification	6	16-bit fixed pt.	AMD Xilinx Zynq
Proposed work	C2Q	32	32-bit floating pt.	AMD Xilinx Zynq

Reported Work	Algorithm	Number of qubits	Precision	FPGA device
Proposed work	QHT	32	32-bit floating pt.	AMD Xilinx X

(N/A ≡ Not Available, QFT ≡ Quantum Fourier Transform, QHT ≡ Quantum Haar Transform, C2Q ≡ Classical-to-Quantum)

저자들의 분석 — 그리고 이 논문 전체의 결론입니다.

기존 연구들의 큐비트 수가 낮은 이유는 **에뮬레이션 기법이 비효율적으로 자원 집약적이었고, 에뮬레이터 설계가 on-chip 자원에 묶여 있었기(bound by on-chip resources)** 때문입니다. 게다가 기존 에뮬레이터들은 **on-board 메모리를 효율적으로 쓰지 않았고, 실험에서 on-board 메모리 사용량을 아예 보고하지도 않았습니다** (표의 해당 열이 전부 N/A인 것을 보세요).

그리고 결정적인 한 문장:

“Classical emulation of quantum algorithms is inherently memory-bound and in our work we are taking full advantage of both available on-chip and on-board memory resources, which enabled us to achieve larger-scale quantum circuit emulation compared to reported work.”

(고전 하드웨어에서의 양자 알고리즘 에뮬레이션은 **본질적으로 메모리에 묶여 있으며(memory-bound)**, 이 연구는 가용한 on-chip 및 on-board 메모리 자원을 모두 최대한 활용함으로써 기존 연구 대비 더 큰 규모의 양자 회로 에뮬레이션을 달성할 수 있었다.)

표를 읽는 법. 3, 3, 7 큐비트 vs 32 큐비트. 이걸 4.5배 차이가 아니라 $2^{25} = 3,300$ 만 배의 상태벡터 크기 차이입니다. 그리고 그 차이를 만든 열은 “FPGA device”가 아니라 **“On-board memory”** 입니다. 기존 연구들이 전부 N/A인 그 열입니다. 정밀도도 16/24-bit **고정소수점(fixed-point)** vs 32-bit **부동소수점(floating-point)** 이고, 동작 주파수도 $80_{250\text{ MHz vs }411} 414\text{ MHz}$ 입니다.

한편 “Emulation time” 열을 순진하게 비교하면 안 됩니다. 기존 연구는 10^{-8} 초, 이 논문은 7.5초입니다. 겉보기엔 이 논문이 8자릿수 느립니다. 그러나 이걸 **3 큐비트(진폭 8개)와 32 큐비트(진폭 43억 개)를 비교하는 것**입니다. 진폭 하나당 시간으로 환산하면 $7.507/2^{32} \approx 1.7\text{ ns/진폭}$ vs $8.4 \times 10^{-8}/2^3 \approx 10.5\text{ ns/진폭}$ 으로 **오히려 이 논문이 6배 빠릅니다**. 표의 마지막 열은 규모가 다른 것들을 나란히 놓은 것이므로 반드시 큐비트 열과 함께 보아야 합니다.

10. 결론과 향후 과제

원문 p.13

저자들의 정리는 다음과 같습니다. 양자 알고리즘의 효율적 에뮬레이션은 양자 컴퓨팅 응용을 연구하기 위해 필요합니다. 이 논문은 양자 알고리즘의 **완전한(complete)** 에뮬레이션을 위한 FPGA 기반 프레임워크를 제시했으며, 프레임워크는 **C2Q 데이터 인코딩용 하드웨어 커널**과 **알고리즘 연산 에뮬레이션용 커널**로 구성됩니다. 이 프레임워크 덕분에 QHT 같은 알고리즘을 조사하고 최적화를 제안할 수 있었으며, C2Q 데이터 인코딩용 최적화 회로도 제시했습니다. HPRC 상에서 C2Q와 QHT를 결합 에뮬레이션했고, 실제

이미지 데이터를 사용한 실험에서 다차원 QHT 후 **정확하고 올바른 데이터 분해**를 확인했습니다. 소프트웨어 에뮬레이터 및 최신 양자 회로 시뮬레이터와의 벤치마크에서 제안 하드웨어 에뮬레이터가 **더 빠르고 더 확장성이 높음**을 보였습니다.

향후 과제 (저자들이 직접 밝힌 것):

1. **메모리 I/O 병목을 줄이는 최적화** — 저자들 스스로 이것이 다음 병목임을 인정합니다.
2. **커널 간 버퍼링(inter-kernel buffering) 개선**.
3. **Q2C(quantum-to-classical) 데이터 판독 기법 연구** — 더 현실적인 완전 양자 알고리즘 에뮬레이션을 위해.

3번이 흥미롭습니다. 이 논문은 “complete”를 표방했지만 실제로는 **입력 측(C2Q)만 완성**했습니다. 출력 측, 즉 상태벡터에서 고전적 답을 뽑아내는 **측정(measurement)** 과정은 여전히 모델링되지 않았습니다. 실제 양자 컴퓨터에서는 측정이 확률적이라 여러 shot을 반복해야 하는데, 에뮬레이터는 진폭 배열을 통째로 볼 수 있어 이 비용을 건너뛸 수 있습니다. **우리 프로젝트의 S_MEAS 상태(argmax로 정답을 뽑는 것)가 정확히 이 문제입니다** — 그건 진짜 측정이 아니라 컨닝입니다.

11. 스케일링 한계 정리 — 무엇이 n 을 묶는가 (해설자 정리)

이 절은 원문의 특정 절을 옮긴 것이 아니라, 원문 곳곳(Table II, Table III, p.4·p.12·p.13)에 흩어진 숫자를 하나로 모아 해설자가 정리한 것입니다. 수치는 모두 원문에서 나왔지만 아래의 산술 전개는 원문에 명시되어 있지 않습니다.

이 논문을 우리 프로젝트에 쓰려면 **딱 하나의 질문에 답할 수 있어야** 합니다: **왜 32 큐비트에서 멈추는가?**

11.1 답: 16 GB DIMM 하나가 정확히 2^{32} 개의 32-bit 진폭입니다

숫자를 맞춰보면 이렇습니다.

진폭 하나가 32-bit 부동소수점이면 4 bytes이므로,

$$2^{32} \times 4 \text{ bytes} = 2^{34} \text{ bytes} = 16 \text{ GiB}$$

Table III의 “On-board memory” 열이 정확히 **16G**입니다. 그리고 가장 큰 입력 이미지가 $(32,768 \times 32,768 \times 4) = 2^{32}$ 픽셀입니다. **우연이 아닙니다**. 양의 실수 데이터에 대해 **DDR4 DIMM 한 개가 딱 32 큐비트를 담습니다**. 33 큐비트를 하려면 32 GiB가 필요한데, DIMM 하나에 안 들어갑니다.

11.2 지수의 벽

큐비트 n	진폭 개수 2^n	실수 32-bit (4 B/진폭)	복소 32-bit (8 B/진폭)	어디에 들어가나
3	8	32 B	64 B	레지스터 몇 개 (기존 FPGA 연구들 [37][38])
7	128	512 B	1 KiB	레지스터 (Lee et al. [39])
10	1,024	4 KiB	8 KiB	BRAM 1개

큐비트 n	진폭 개수 2^n	실수 32-bit (4 B/진폭)	복소 32-bit (8 B/진폭)	어디에 들어가나
16	65,536	256 KiB	512 KiB	BRAM 수십 개 — Qiskit initialize() 한계
20	1.05×10^6	4 MiB	8 MiB	on-chip 한계 근처 — Qiskit + Method 2 한계
24	1.68×10^7	64 MiB	128 MiB	on-chip 불가, DDR 필수
28	2.68×10^8	1 GiB	2 GiB	DDR — Qiskit QHT 한계
30	1.07×10^9	4 GiB	8 GiB	DDR
32	4.29×10^9	16 GiB	32 GiB	DIMM 1개 정확히 — 이 논문의 한계
34	1.72×10^{10}	64 GiB	128 GiB	보드 전체 4개 DIMM
38	2.75×10^{11}	1 TiB	2 TiB	QuEST의 8 TiB 슈퍼컴 한계 [33]
40	1.10×10^{12}	4 TiB	8 TiB	슈퍼컴퓨터
50	1.13×10^{15}	4 PiB	8 PiB	존재하지 않음

큐비트 1개 추가 = 메모리 2배. 이 표에서 읽어야 할 것은 세 가지입니다.

첫째, **논문이 32에서 멈춘 건 설계 실패가 아니라 물리 법칙입니다.** LUT를 9.48%밖에 안 쓰고도 못 올라간 이유가 이겁니다. 칩을 10배 큰 걸로 바꿔도 큐비트는 **3개** 늘어납니다 ($2^3 = 8 \approx 10$).

둘째, **on-chip에 상태벡터를 두는 순간 20 큐비트 남짓이 천장입니다.** 원문은 동적 영역의 BRAM을 “2,000 × 36KB”로 적었는데, Xilinx BRAM 블록은 관례적으로 36 Kbit(= 4.5 KB)이므로 실제 총량은 약 **9 MB**로 보는 것이 맞습니다(원문 표기를 글자 그대로 36 KB로 읽으면 72 MB). 어느 쪽으로 읽든 결론은 같습니다 — on-chip BRAM만으로는 **대략 21~24 큐비트**가 한계이고, 그 위로는 **on-board DDR4 외에 선택지가 없습니다.** 기존 연구들이 3~7 큐비트에 머문 건 상태벡터를 BRAM도 아닌 레지스터에 뒀기 때문입니다.

셋째, **모든 경쟁자가 같은 벽에 부딪힙니다.** 12 GB GPU → 29 큐비트, 16 GB GPU → 29 큐비트, 500 GB Qiskit → 28 큐비트(QHT), 16 GB DIMM → 32 큐비트, 8 TiB 슈퍼컴 → 38 큐비트. 플랫폼이 뭐든 **큐비트 수는 메모리 용량의 로그**, 즉 $n \approx \log_2 M$ 입니다. FPGA가 이긴 건 큐비트 수가 아니라 **같은 큐비트 수에서의 속도와 비용**입니다.

11.3 그럼 FPGA가 이긴 것은 정확히 무엇인가

정직하게 정리하면 FPGA의 승리는 **세 가지**이고, 셋 다 큐비트 수가 아닙니다.

1. **비용/전력당 성능:** 500 GB 서버(Qiskit)가 28 큐비트에서 하는 일을, 16 GB 보드 한 장이 32 큐비트로 더 빠르게 합니다. 20 큐비트 C2Q에서 4자리수 speedup.
2. **낮은 상수 계수:** 지수 곡선의 기울기는 못 바꾸지만 y 절편을 CPU 대비 $\times 3.49 \sim \times 21.66$ 낮춥니다.
3. **결정론적 파이프라인:** 캐시 미스나 GC 없이 매 클럭 진폭을 처리합니다. 그래서 큰 n 에서 CPU를 이깁니다 (작은 n 에서는 캐시가 있는 CPU가 이기고요).

11.4 이 논문이 하지 않은 것 — 다중 FPGA

우리 스터디 맥락에서 오해하기 쉬운 부분이라 명확히 해둡니다. 제목에 “High-Performance Reconfigurable Computers”가 있고 “Scalable”이 있지만, **이 논문에는 다중 FPGA도, 다중 노드도, FPGA 간 통신도 없습니다.** 여기서 HPRC는 **호스트 1대 + 가속기 보드 1장**을 뜻합니다. 병렬성은 전부 칩 내부입니다: fully pipelined

커널, fine-grain 병렬성, 그리고 (미실현) 커널 $\times 10$ 복제. XCU250이 4개의 SLR을 가진다는 언급은 있지만 SLR 간 분산 전략은 다루지 않습니다.

저자들이 다중 노드를 다루는 유일한 대목은 **관련 연구를 비판할 때**입니다 — 분산 시뮬레이터는 “서버 간 통신 오버헤드가 무시 못 할 수준이며 총 실행 시간에 더해진다”(원문 p.3). 그리고 이건 원리적으로 옳은 지적입니다. 상태벡터를 k 개 노드에 쪼개면 큐비트가 $\log_2 k$ 개 늘지만, 상위 큐비트에 게이트를 걸 때마다 **노드 간 전량 통신(all-to-all)** 이 필요합니다. 즉 노드를 1000배 늘려야 큐비트 10개를 얻고, 그 대가로 통신이 지배합니다.

따라서 이 논문에서 “다중 FPGA 병렬화 설계”를 배울 것을 기대하면 안 됩니다. 배울 것은 단일 FPGA에서 메모리 계층을 어떻게 써야 하는가입니다.

12. 이 프로젝트에 어떻게 쓰이나

우리 저장소에는 project1/hardware/grover_ip.v, grover_min_ip.v, grover_tb.v 가 있습니다. 현재 상태를 이 논문의 좌표계에 올려놓고, 무엇을 가져오고 무엇을 버릴지 정리합니다.

12.1 현재 우리 IP의 좌표

grover_ip.v 는 $N=4$, NBITS=2, 즉 **2 큐비트**입니다. grover_min_ip.v 는 $N=8$, NBITS=3, **3 큐비트**입니다. Table III에서 Khalid(2004)와 Aminian(2008)의 그로버가 정확히 **3 큐비트**입니다. 즉 우리는 지금 **2004년 수준에 서 있고, 그건 시작점으로 완벽히 정상**입니다.

우리 IP의 구조를 논문 용어로 번역하면 이렇습니다.

우리 코드	논문 용어	평가
reg signed [DW-1:0] amp [0:N-1] INIT_AMP 상수로 균등중첩	상태벡터, on-chip 레지스터 배열 C2Q 없음 (초기화 가정)	Table III의 “On-board memory: N/A” 논문이 “incomplete”라 비판하는 바로
S_ORACLE: amp[MARK] <= -amp[MARK]	오라클(oracle) U_f	진폭 부호 반전 — 올바른
S_SUM + S_DIFF: 2*mean - a	확산(diffusion) U_ψ	아래 12.3 참고 — 잘한 부분
S_MEAS: 진폭 절댓값이 최대인 인덱스 선택	측정(measurement)	논문의 미해결 과제(Q2C)와 동일
DW=18, FRAC=16 (Q1.16)	고정소수점(fixed-point)	논문이 “low accuracy”라 비판하는 그

12.2 가져올 것 (1) — 상태벡터를 레지스터에서 메모리로

이것 하나가 이 논문에서 가져올 가장 중요한 교훈입니다.

현재 코드는 이렇습니다.

```
reg signed [DW-1:0] amp [0:N-1]; // N개 진폭 전부 온칩
...
S_DIFF: begin
    for (i=0;i<N;i=i+1) amp[i] <= two_mean - amp[i]; // N개를 1클럭에 전부
end
```

이 for 루프는 합성 시 **완전히 펼쳐집니다(unrolled)**. N 개의 뿔셈기가 물리적으로 생기고 N 개의 레지스터가 동시에 갱신됩니다. $N = 8$ 이면 예쁘고, $N = 1024$ (10 큐비트)면 뿔셈기 1024개가 생기며, $N = 2^{20}$ 이면 합성이 죽습니다. **회로 크기가 2^n 으로 증가합니다.** 이것이 Table III의 기존 연구들이 3~7 큐비트에 갇힌 정확한 이유이고, 논문 표현으로 “emulator designs were bound by on-chip resources”입니다.

논문의 처방은 정반대입니다. **커널을 작게 고정하고 데이터를 흘립니다.** 3D-QHT 커널이 BRAM 2개, LUT 0.71%로 32 큐비트를 도는 이유가 이겁니다. 우리 식으로 옮기면:

```
// 개념 스케치 — 회로 크기가 N과 무관해집니다
// 상태벡터는 BRAM/DDR에 있고, 한 번에 몇 개씩만 읽어서 처리
for each chunk of amplitudes:      // 순차적 스트리밍
    read amp[i] from memory
    compute 2*mean - amp[i]        // 뿔셈기 1개 (또는 P개)
    write amp[i] back to memory
```

이렇게 하면 N 이 커져도 하드웨어는 그대로이고, 실행 시간만 $O(N)$ 으로 늘어납니다. 논문의 Fig. 8이 로그 축에서 직선인 이유가 정확히 이것입니다.

단, 확산 연산에는 **평균**이 필요해서 전체 합을 먼저 알아야 합니다. 스트리밍으로 하면 **2-pass**가 됩니다: pass 1에서 진폭을 훑으며 합을 누적하고 평균을 구한 뒤, pass 2에서 다시 훑으며 $2\bar{a} - a_i$ 를 씁니다. 메모리를 두 번 읽는 대가로 회로 크기가 n 과 무관해집니다. 이게 남는 장사입니다.

12.3 가져올 것 (2) — 우리는 이미 “커널 기반”을 하고 있습니다

논문의 Fig. 5는 그로버를 **dense 행렬** → **CMAC** 진영으로 분류합니다. 순진하게 읽으면 “그로버는 $N \times N$ 행렬 곱이니 $O(4^n)$ ” 이 되어 절망적입니다.

그런데 우리 코드는 이미 그 함정을 피해 있습니다. 확산 연산자 $U_\psi = 2|s\rangle\langle s| - I$ 는 행렬로 쓰면 조밀하지만, 그 작용은

$$a_i \longrightarrow 2\bar{a} - a_i, \quad \bar{a} = \frac{1}{N} \sum_j a_j$$

로 $O(N)$ 에 끝납니다. 이것이 바로 S_SUM + S_DIFF 입니다. 오라클도 부호 반전이라 $O(N)$ 입니다. 즉 **우리 그로버 한 iteration은 $O(N) = O(2^n)$ 이지 $O(4^n)$ 이 아닙니다.** 논문의 QHT 커널과 같은 복잡도 등급입니다.

게다가 N 이 2의 거듭제곱이라 $\text{mean} = \text{sum} \ggg \text{NBITS}$ 로 나눗셈이 시프트가 됩니다 — 이걸 이미 좋은 하드웨어 감각입니다. 그리고 표준 그로버는 진폭이 **실수로만** 유지되어(grover_ip.v 주석에 정확히 지적되어 있습니다) 복소수가 필요 없습니다. 이걸 **메모리를 절반으로 줄이는** 대단히 큰 이점입니다 — 11.2절 표에서 “실수 4 B” 열이 “복소 8 B” 열보다 큐비트 1개를 더 벌여줍니다.

결론: 자료구조만 메모리로 옮기면 우리 알고리즘 구조는 이미 논문 수준입니다. CMAC은 우리에게 필요 없습니다.

12.4 가져올 것 (3) — 수 표현: 우리 Q1.16은 언제 죽는가

논문은 32-bit 부동소수점(floating-point)을 쓰고 기존 연구의 고정소수점(fixed-point)을 “low accuracy”라 깎아내립니다. 우리 IP는 Q1.16(FRAC=16)입니다. **언제 문제가 되는지 정확히 계산해 봅시다.**

그로버의 초기 진폭은 $1/\sqrt{N} = 2^{-n/2}$ 입니다. Q1.16에서 이 값의 정수 표현은:

$$\text{INIT_AMP} = \frac{1}{\sqrt{2^n}} \times 2^{16} = 2^{16-n/2}$$

검산해 보면 코드와 정확히 맞습니다. $n = 2$: $2^{15} = 32768$ (grover_ip.v 의 INIT_AMP=18'sd32768).
 $n = 3$: $2^{14.5} = 23170$ (grover_min_ip.v 의 INIT_AMP=20'sd23170).

이제 n 을 키우면:

큐비트 n	INIT_AMP = $2^{16-n/2}$	남은 정밀도
2	32,768	15 bits — 넉넉
8	4,096	12 bits — 양호
16	256	8 bits — 슬슬 불안
20	64	6 bits — 위험
24	16	4 bits — 사실상 붕괴
28	4	2 bits
32	1	0 bits — 진폭이 곧 1 LSB

Q1.16 고정소수점은 $n \approx 20$ 근처에서 무너집니다. 32 큐비트에서는 초기 진폭이 **정확히 1 LSB**가 되어 아무 의미가 없습니다. 그로버는 진폭을 조금씩 증폭하는 알고리즘인데, 증폭할 단위 자체가 양자화 오차와 같아지면 알고리즘이 동작하지 않습니다.

그래서 논문이 **32-bit 부동소수점을 쓴 겁니다.** 부동소수점은 지수부가 따로 있어 2^{-16} 이든 2^{-100} 이든 24-bit 가수(mantissa)의 상대 정밀도를 유지합니다. 정리하면 우리에게 **두 개의 천장**이 있습니다:

- 수 표현의 천장: $Q1.16 \rightarrow n \approx 20$
- 메모리의 천장: 16 GB DDR4 $\rightarrow n = 32$

수 표현 천장이 먼저 옵니다. 그러므로 20 큐비트를 넘길 계획이라면 부동소수점(또는 최소 Q1.30 이상의 넓은 고정소수점, 혹은 블록 부동소수점)으로 가야 합니다. 반대로 말하면 **20 큐비트 이하를 목표로 하는 한 Q1.16으로 충분**하고, 이는 이미 Table III의 어떤 기존 연구보다도 훨씬 큰 규모입니다.

12.5 가져올 것 (4) — 정직한 벤치마크 방법론

논문의 측정 방식은 그대로 베낄 가치가 있습니다.

- $T_{FPGA} = T_{in} + T_{comp} + T_{out}$ 로 **데이터 전송을 포함**해 보고할 것. 커널 시간만 자랑하는 건 부정직합니다.
- T_{setup}, T_{config} (비트스트림 로딩)는 **제외**하되 **제외했다고 명시**할 것.

- CPU 레퍼런스 구현을 반드시 만들 것 (논문은 C++로 만들었습니다). 검증(verification)과 벤치마크 양쪽에 필요합니다. grover_tb.v 만으로는 부족합니다.
- 교차점을 찾아 보고할 것. 논문은 “CPU가 12 큐비트까지는 더 빠르다”고 정직하게 씁니다. 우리도 작은 n 에서는 CPU가 이길 것이고, 그 사실을 숨기면 안 됩니다. 오히려 교차점의 존재가 결과의 신뢰도를 높입니다.

12.6 범위 밖 — 하지 말아야 할 것

한정된 학부 인턴 기간을 지키기 위해, 이 논문에서 가져오지 말아야 할 것을 분명히 합니다.

- C2Q 인코딩 (Method 1 / Method 2). 논문 지면의 절반이 여기지만 우리에게는 필요 없습니다. 그로버는 임의의 데이터를 실지 않습니다. 균등중첩 $H^{\otimes n}|0\rangle^{\otimes n}$ 에서 시작하는데, 그건 모든 진폭이 $1/\sqrt{N}$ 인 상수입니다. 현재 코드의 INIT_AMP 상수 한 줄이 이미 완결된 정답입니다. 논문이 지적하는 “incomplete” 비판은 임의 데이터를 실는 알고리즘(QHT, 양자 이미지 처리)에만 해당하며 그로버에는 해당하지 않습니다. Table I 전체를 이해하려 애쓰지 마세요. Method 1이 “에뮬레이션이라면 비유니터리를 써도 된다”는 발상만 챙기면 충분합니다.
- QHT, 하르 웨이블릿, 순열(RoL/RoR), Eq. 3-9, 21-22. 전부 QHT 전용입니다. 그로버에는 순열 네트워크가 없습니다.
- 다중 FPGA / 다중 노드 확장. 11.4절에서 봤듯 논문에 없습니다. 그리고 단일 보드로 32 큐비트가 가능한데 다중 보드를 고민하는 건 순서가 틀렸습니다.
- 커널 $\times 10$ 복제. 논문의 미실험 제안일 뿐이고, throughput은 올리되 큐비트는 1개도 못 올립니다.
- 32 큐비트 목표. 우리 IP에서 현실적 다음 목표는 10~16 큐비트입니다. 3 → 16 큐비트도 상태벡터가 8개에서 65,536개로 8,000배 커지는 것이고, 그 과정에서 12.2절의 아키텍처 전환(레지스터 → BRAM 스트리밍)을 반드시 겪게 됩니다. 그 전환을 해내는 것만으로 충분히 좋은 인턴 성과입니다.

12.7 다음에 읽을 것

- [43] N. Mahmud, E. El-Araby, D. Caliga, “Scaling reconfigurable emulation of quantum algorithms at high precision and high throughput,” Quantum Engineering, vol. 1, no. 2, 2019, Art. no. e19. — Fig. 5가 그로버를 CMAC 진영으로 분류하며 지목하는 바로 그 참고문헌입니다. 같은 연구실(El-Araby)의 선행 연구이고 제목에 “Scaling”, “high precision”, “high throughput”이 다 들어 있습니다. 그로버 FPGA 에뮬레이터를 만든다면 이 논문이 이 문서보다 더 직접적입니다.
- [39] Y. H. Lee, M. Khalil-Hani, M. N. Marsono, “An FPGA-based quantum computing emulation framework based on serial-parallel architecture,” Int. J. Reconfigurable Computing, 2016. — Table III에서 7 큐비트 그로버를 한 연구로, 기존 최고 기록입니다. 우리의 직접적인 비교 대상이자 현실적인 단기 목표선입니다.

12.8 한 문단 요약

이 논문에서 우리 프로젝트가 가져갈 것은 문장 하나로 줄어듭니다: “고전 하드웨어에서의 양자 에뮬레이션은 본질적으로 memory-bound다.” 우리 grover_ip.v 가 3 큐비트에 갇힌 이유는 코드가 나빠서가 아니라 상태벡터를 레지스터에 뒀기 때문이고, 논문이 32 큐비트에 도달한 이유는 그걸 DDR4로 옮겼기 때문이며, 논문이 32에서 멈춘 이유는 16 GB DIMM 하나가 정확히 2^{32} 개의 32-bit 진폭이기 때문입니다. 로직은

처음부터 끝까지 병목이 아니었습니다 (LUT 9.48%). 우리가 할 일은 오라클과 확산을 더 예쁘게 짜는 게 아니라, 진폭 배열을 메모리로 내보내고 커널을 스트리밍 파이프라인으로 바꾸는 것입니다.